

Activity 3: SQL Subqueries

Instructions:

- Use the **ecommerce.db** database to complete the following queries.
- Work with people at your table to solve these queries, but each person must submit their own work individually.
- Write your SQL queries directly in this Word document under each question and submit the completed file on Canvas by the end of class.

1. Find all products that are more expensive than the average product price. Show product name and current price.

```
SELECT product_name, current_price
FROM products
WHERE current_price > (SELECT AVG(current_price) FROM products);
```

2. Find products that have received at least one review with a rating of 4 or higher. Show product name and current price. Use a subquery with EXISTS.

```
SELECT p.product_name, p.current_price
FROM products p
WHERE EXISTS (SELECT 1 FROM reviews r WHERE r.product_id = p.product_id AND
r.rating >= 4);
```

3. Using a CTE, find all orders with above-average total amounts. Then join with customers to show order ID, customer name, and order total. Order by total amount (highest first).

```
SELECT totals.order_id, c1.first_name, c1.last_name, total_amount FROM
(SELECT * FROM orders o1 WHERE total_amount >
(SELECT avg(total_amount) FROM orders)
ORDER BY total_amount) AS totals
INNER JOIN customers c1 ON c1.customer_id = totals.customer_id ORDER BY
total_amount DESC;
```

4. Using multiple CTEs, first calculate the average order total, then find customers whose average order amount is above the overall average. Show customer ID, customer name, their average order amount, and number of orders.

```
WITH overall_avg AS (  
    SELECT AVG(total_amount) AS avg_total  
    FROM orders  
)  
customer_avgs AS (  
    SELECT  
    customer_id,  
    AVG(total_amount) AS avg_order_amount,  
    COUNT(*) AS order_count  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT  
c.customer_id, c.first_name, c.last_name, ca.avg_order_amount, ca.order_count  
FROM customer_avgs ca  
JOIN customers c ON ca.customer_id = c.customer_id  
JOIN overall_avg oa ON ca.avg_order_amount > oa.avg_total  
ORDER BY ca.avg_order_amount DESC;
```

5. Using a CTE, calculate each customer's total spending. Then find customers who have spent more than \$5000 total. Show customer ID, first name, last name, and total spent.

```
WITH customer_totals AS (  
    SELECT customer_id, SUM(total_amount) AS total_spent  
    FROM orders  
    GROUP BY customer_id)  
SELECT c.customer_id, c.first_name, c.last_name, ct.total_spent  
FROM customer_totals ct  
JOIN customers c ON ct.customer_id = c.customer_id  
WHERE ct.total_spent > 5000  
ORDER BY ct.total_spent DESC;
```